



Autor:	Data de Criação: 01/10/2025
Pedro Vale de Souza - 231038733	Última Atualização: 23/10/2025

Documento da Arquitetura de Software

INFORMAÇÕES SOBRE O PROJETO

Este é um documento referente ao projeto da matéria de Engenharia de Software (CIC0105), que tem como foco a criação de um produto para auxiliar na aplicação da metodologia Scrum no processo de desenvolvimento de software denominado “Sistema Scrum”.

INFORMAÇÕES SOBRE ESSE DOCUMENTO

Este documento, Arquitetura de Software, visa destacar o estilo e visão de arquitetura, e também descrever os componentes e seus relacionamentos para a realização do sistema. Por esse documento será encontrado, o objetivo da arquitetura, as ferramentas com suas dependências, os requisitos de qualidade oferecidos, estrutura do padrão de arquitetura, mecanismos de implementação e as perspectivas de organização da arquitetura.

Os responsáveis pela realização do trabalho

Quadro 01 - Nome dos Integrantes do Grupo e Matrícula

Nome	Matrícula
Gabriel Francisco de Oliveira Castro	202066571
Lucas Issamu Hashimoto	231003504
Pedro Vale de Souza	231038733
Matheus Duarte da Silva	211062277
Suyanne Sara Miranda Silva	222006445

Fonte: Do Autor



HISTÓRICO DE REVISÃO

Esse trecho contém todo o histórico de revisão e mudanças nesse documento, para garantir um controle e um rastreio da evolução do documento.

Quadro 02 - Histórico de Revisão

Data	Autor	Descrição	Versão
01/10/2025	Pedro Vale de Souza	Criação do documento	v0.1.0
03/10/2025	Pedro Vale de Souza	Refatoração do documento	v0.2.0
05/10/2025	Pedro Vale de Souza	Refatoração textos do documento	v0.3.0
09/10/2025	Pedro Vale de Souza	Refatoração e adição de descrição de domínios	v0.4.0
23/10/2025	Pedro Vale de Souza	Adicionando novas descrições	v0.5.0
29/10/2025	Pedro Vale de Souza	Alterações das descrições	v0.6.0

Fonte: Do Autor

1. Objetivos da Arquitetura

Este documento tem como objetivo descrever a arquitetura do projeto de gestão de equipes baseado no Scrum denominado “Sistema Scrum”. A estrutura deve manter uma base bem definida do back-end, que irá interagir com o front-end por meio de um API RESTful.

Os princípios enfocados neste projeto foram, principalmente:

- **Manutenibilidade:** Facilitar nas modificações futuras e correção de bugs por subdividir funcionalidade em camadas.
- **Simplicidade:** Permitir uma construção simples da arquitetura, provocando um maior entendimento a partir do que cada camada tem de funcionalidade.
- **Integridade dos Dados:** Garantir que as informações sejam armazenadas e recuperadas, com objetivo de assegurar consistências e evitar perdas de dados.

2. Conteúdos do Projeto (design)

- **Interface Web (front-end):** A interação do usuário com o sistema será feita por interface gráfica web, que será possível ser acessada por qualquer dispositivo desktop que tenha acesso à internet.
- **Comunicação via API RESTful (back-end):** A comunicação do front-end para o back-end vai ser por rotas, por protocolo HTTPS seguindo padrão RESTful, que vão permitir troca de dados entre requisições e respostas.
- **Banco de Dados Relacional:** Os dados do sistema serão guardados em uma estrutura relacional de banco de dados, por meio de tabelas já definidas e implementadas pelo back-end do sistema
- **Tecnologias Base:** As ferramentas definidas que vão ser utilizadas na arquitetura são o python como base no back-end e usando Flask (microframework web), no frontend, Nuxt como framework para a implementação da interface, na persistência o SQLAlchemy para iteração das tabelas e miniconda como gerenciamento do ambiente.

3. Dependências de Ferramentas

Restrição	Ferramentas
Linguagem	Python 3.13.5 e JavaScript
Framework	Flask 3.1.2, Nuxt 4.1.2, Vue 3.5.21 e SQLAlchemy Core 2.0.43
Banco de Dados	psycopg2 2.9.10 e Sqlite3 3.50.4
Plataforma	Web

4. Requisitos da Arquitetura

- **Performance:** O banco de dados deve permitir consultas rápidas e eficientes, por meio de otimizações feitas pelo framework SQLAlchemy Core que permitem queries SQL mais velozes e gerenciamento de conexões.
- **Segurança:** O sistema deve permitir segurança ao usuário por meio de autenticação de usuários, validação de acesso, garantindo confidencialidade e integridade das informações.
- **Usabilidade:** A interface do sistema deve ser intuitiva e amigável, incluindo manuais para orientar o usuário, facilitando a operação para todos os públicos.
- **Confiabilidade:** O sistema deve manter a integridade dos dados. A presença de tratamento de erros na camada de persistência permite alertar com dados incorretos.
- **Portabilidade:** O sistema deve ser compatível com diferentes navegadores por meio de uma API RESTful que permite uma comunicação com navegadores web.

5. Representação da Arquitetura

A Arquitetura utilizada no back-end do projeto será baseada em camada (Layers), padrão que promove uma separação entre responsabilidades, recebidas diretamente da interação com o usuário na interface (UI). Nesse caso, o back-end será organizado em duas camadas bem definidas, que se comunicam sequencialmente para obter requisições que o usuário faz na interface consultando sempre no Banco de Dados.

Optou-se por essa arquitetura devido à sua capacidade de separar responsabilidades, permitindo distinguir as requisições HTTP (camada de apresentação) das consultas SQL (camada de persistência). Além disso, esse estilo de arquitetura facilita a manutenção do código ao promover a coesão, isto é, cada camada possui funções específicas, bem definidas e relacionadas, tornando o sistema mais organizado e propício à evolução.

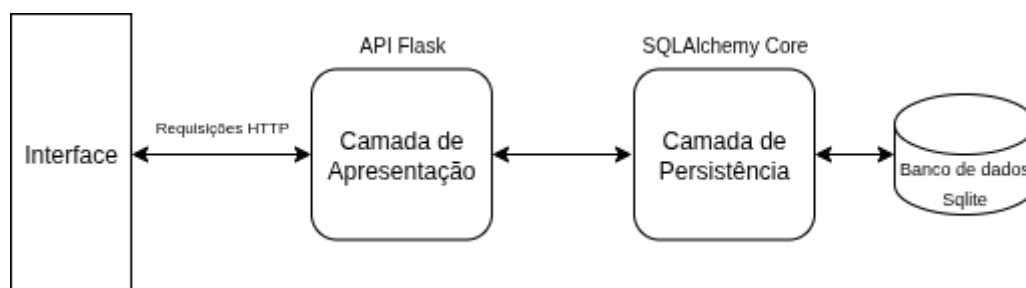
O paradigma de programação adotado foi o procedural, pois o Python possui diversas bibliotecas que abstraem funcionalidades. Por exemplo, o SQLAlchemy

Core pode refletir tabelas, eliminando a necessidade de criar classes e objetos para o ORM.

5.1 Estrutura das Camadas

- Camada de Apresentação : Essa é a primeira camada da arquitetura, responsável pela comunicação entre o sistema e o usuário. Ela recebe as requisições, valida os dados iniciais, os armazena temporariamente e os encaminha para a camada de serviço. Seu papel é estabelecer os canais de comunicação por meio das rotas e interagir com a interface do front-end.
- Camada de Persistência : Camada mais interna da arquitetura que é responsável pela comunicação com o Banco de Dados. Sua função é consultar e resgatar dados para provê-los à camada de serviço.

5.2 Diagrama da Arquitetura



6. Mecanismos da Arquitetura

6.1 Rotas e Manipulação de Requisitos HTTP (FLASK)

As requisições HTTP são gerenciadas por rotas pelo framework do Flask. Essas rotas fazem a comunicação da interface com o sistema. A biblioteca também faz manipulações de acesso aos dados da requisição, por JSON, feito através do objeto *request* do Flask, e as respostas são formatadas como JSON usando *jsonify*.

A API abstrai as comunicações com o front-end, que podem ser manipulados através de URLs e métodos HTTP padrão (GET, POST, PUT, DELETE).

6.2 Gerenciamento de Conexões e Reflexão de Tabelas do Banco de Dados (SQLAlchemy CORE)

A interação com o banco de dados é realizada através do SQLAlchemy Core. O mecanismo central (Engine), gerencia as conexões para otimizar o acesso ao banco. As queries SQL são construídas usando o Core Expression Language (*insert, read, update, delete*), que abstraem comandos SQLs puros, sobre objetos obtidos via reflexão.



A inicialização da aplicação para ler (refletir) a estrutura das tabelas (colunas, tipos, chaves) diretamente do banco, criando objetos Table, em memória que representam a estrutura real, permitindo que os repositórios construam queries sem definir o schema no código Python.

7. Visões da Arquitetura de Software

7.1 Visão Lógica

Domínios de Negócio

Nome dos Domínios	Formato Específico
Nome	Texto com até 20 caracteres. Cada caractere pode ser números (0-9), letra (A-Z, a-z) e espaços, sem acentuações.
Descrição e Revisão	Texto com até 255 caracteres. Sem restrições
Objetivo e Meta	Texto com até 100 caracteres. Sem restrições
Email	Formato : <i>parte@dominio</i> A <i>parte</i> pode conter letras (A-Z,a-z), dígitos (0-9), pontos (.) ou hífens (-). Pontos e Hífens sempre seguidos por letra ou dígito. O <i>domínio</i> é separado por partes separadas por ponto (.) e não podem começar com pontuações. Comprimento de até 64 caracteres
Senha	De 8 até 16 caracteres permitidos. Cada caractere pode ser dígito, letra e caracteres especiais (!@#\$%&*) . Existe pelo menos uma letra Maiúscula e uma letra Minúscula. Existe pelo menos um dígito Existe pelo menos um caractere especial
CPF	Formato : XXX.XXX.XXX-XX Composto apenas por números
Papel	Dono do Produto, Scrum Master, Desenvolvedor
Estado da Tarefa	Para Fazer, No Processo, Em análise e Terminado

Prazo	Formato : AA/MM/DD
-------	--------------------

Entidades Conceituais

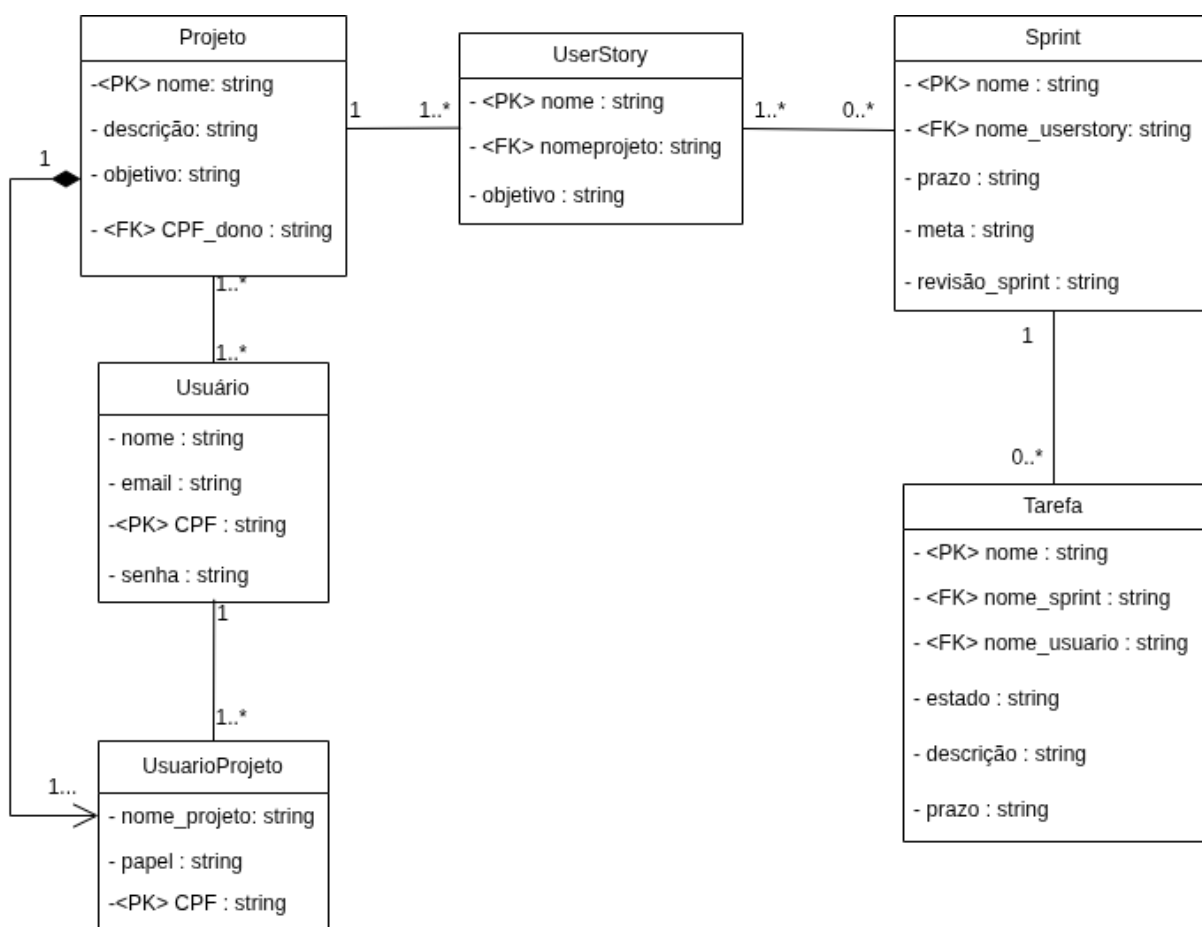


Figura 01: Diagrama de classes do sistema representando as entidades conceituais.

Nota: lembrando que não estão sendo utilizadas classes e objetos na implementação do projeto. Este diagrama é apenas para o entendimento de “blocos lógicos” que são conceituais.

7.2 Visão de Desenvolvimento

Diagrama de Pacotes

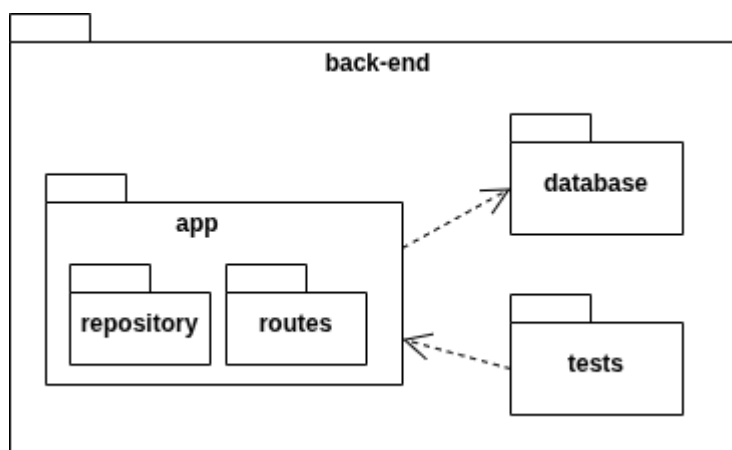


Figura 02: Diagrama de pacotes do sistema.

- **../app/repository** : Implementa a camada de persistência, onde cada módulo das tabelas são repositórios, responsáveis pelo CRUD para cada “entidades conceituais”. Também, contém o módulo de gerenciamento da conexão com a tabela no banco de dados.
- **../app/routes**: Implementa a camada de apresentação, onde cada módulo são rotas dos repositórios.
- **../database**: Dentro desse diretório há a estrutura do banco de dados, e seus scripts complementares.
- **../tests** : Responsável por arquivos que realizam os testes dos arquivos dentro de app. Dentro desse pacote possui os testes de integração dos repository com as routes.

7.3 Visão Física

A Implantação dos componentes de infraestrutura do sistema é composto por infraestrutura de máquina local, onde o back-end, banco de dados e a interface, rodam em arquivos locais, com interação máquina servidor via localhost da API do back-end.

7.4 Casos de Uso

Cenários de Casos de Uso

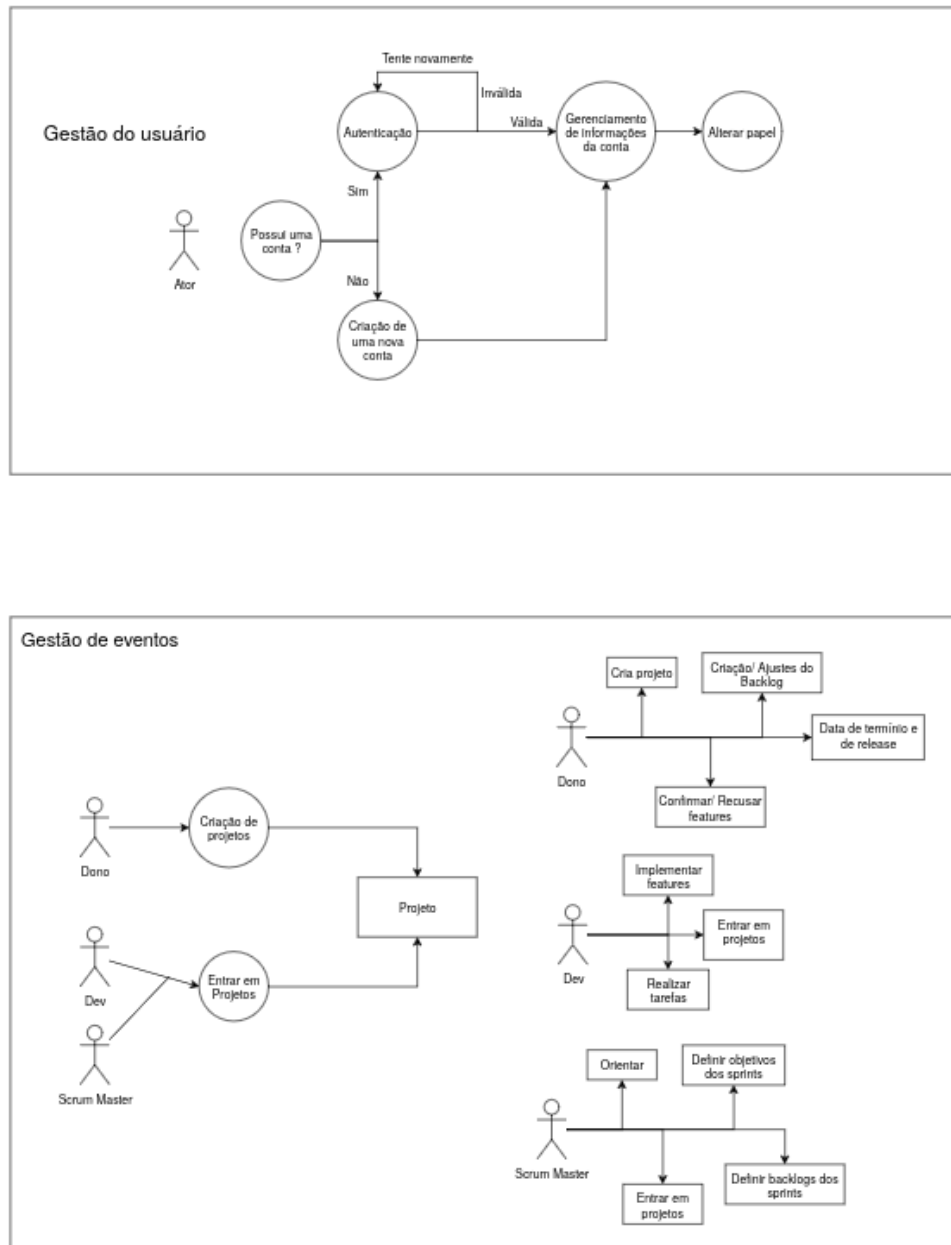


Figura 03: Modelo de casos de uso e relacionamento entre atores.

Nota: Casos de uso representados de forma conceitual, apenas para um breve entendimento.



8. Impacto das Ferramentas Utilizadas

Flask: A simplicidade do Flask em definir rotas (`@app.routes`) e usar Blueprints facilitou a criação de uma API RESTful, que é a forma de comunicação definida com o frontend. A responsabilidade da camada de apresentação ficou claramente delimitada à gestão das requisições e respostas HTTP.

SQLAlchemy Core: Permitiu interagir com o banco via Python sem ORM completo. A função de *reflexão* foi crucial para usar o schema existente. Exigiu a implementação manual do *mapeamento* de dados (dicionários/Rows/respostas).

Conda: Garante a reprodutibilidade do ambiente de desenvolvimento através do `environment.yml`.

SQLite3: No banco de dados (evidenciada pelos scripts) simplificou a configuração do ambiente de desenvolvimento, por ser um banco de dados baseado em arquivo único, sem a necessidade de um servidor separado.